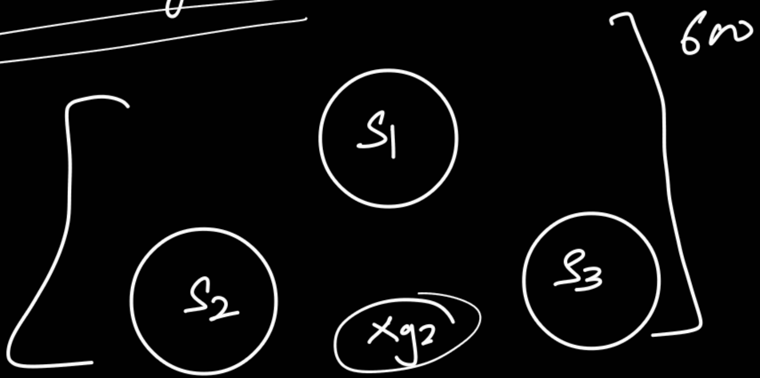
Video-1 it helps in coordinating multiple server in cluster.

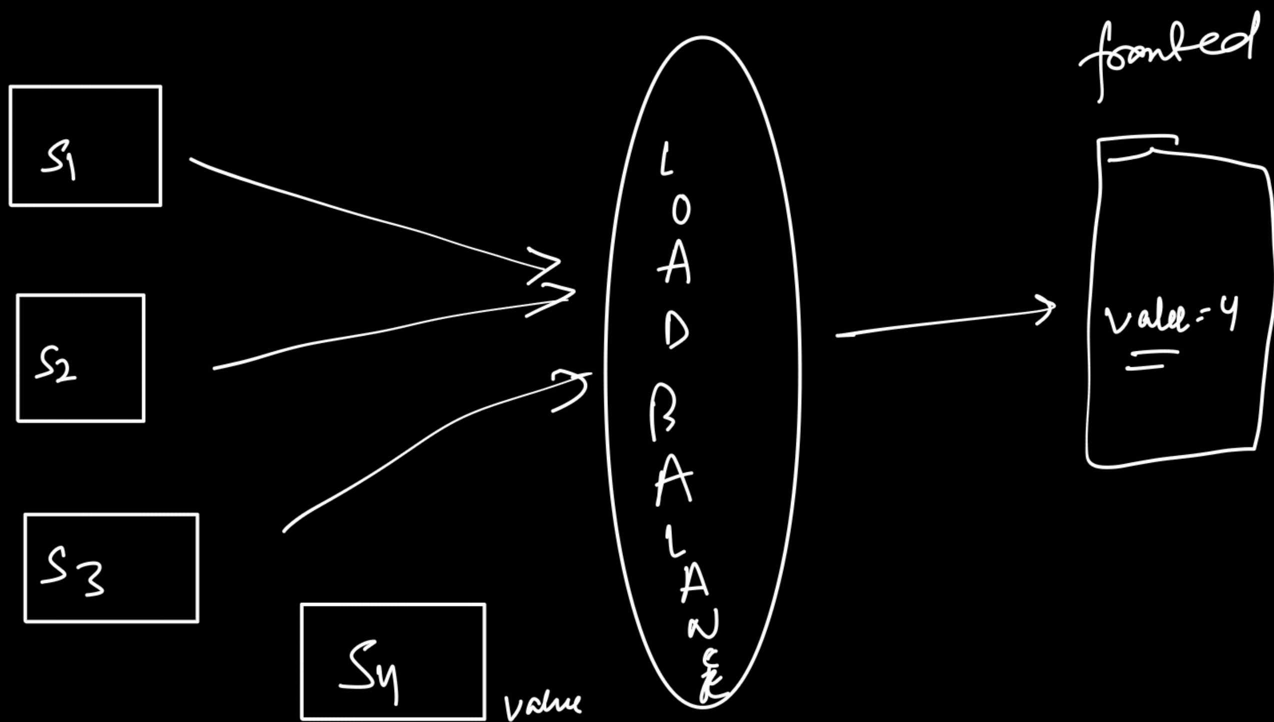
Distributed System :-



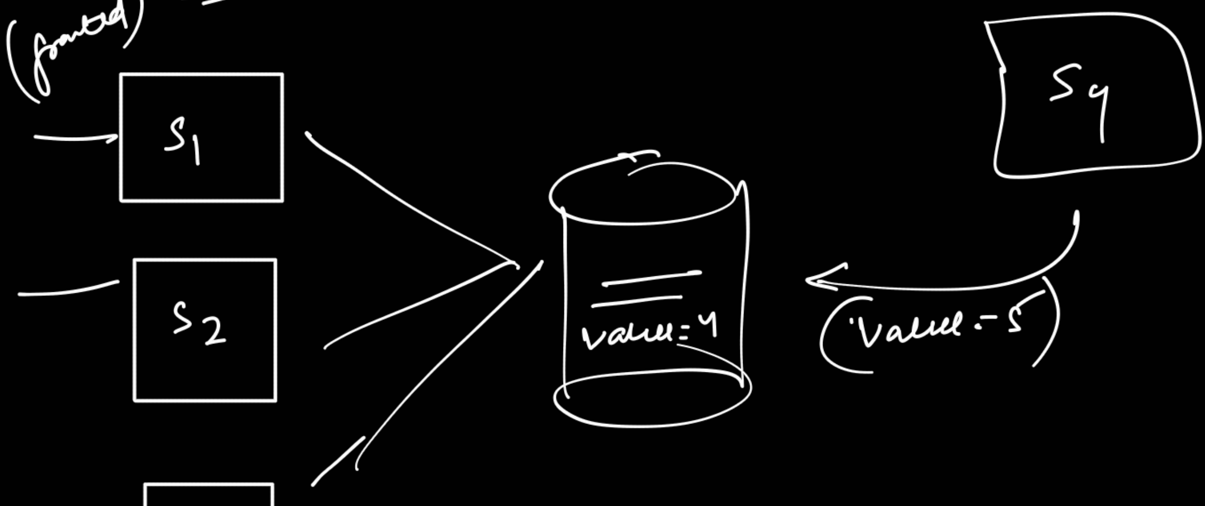
Problems :-

1) Coordination
2) work-load
3) leader election

Configuration Management System



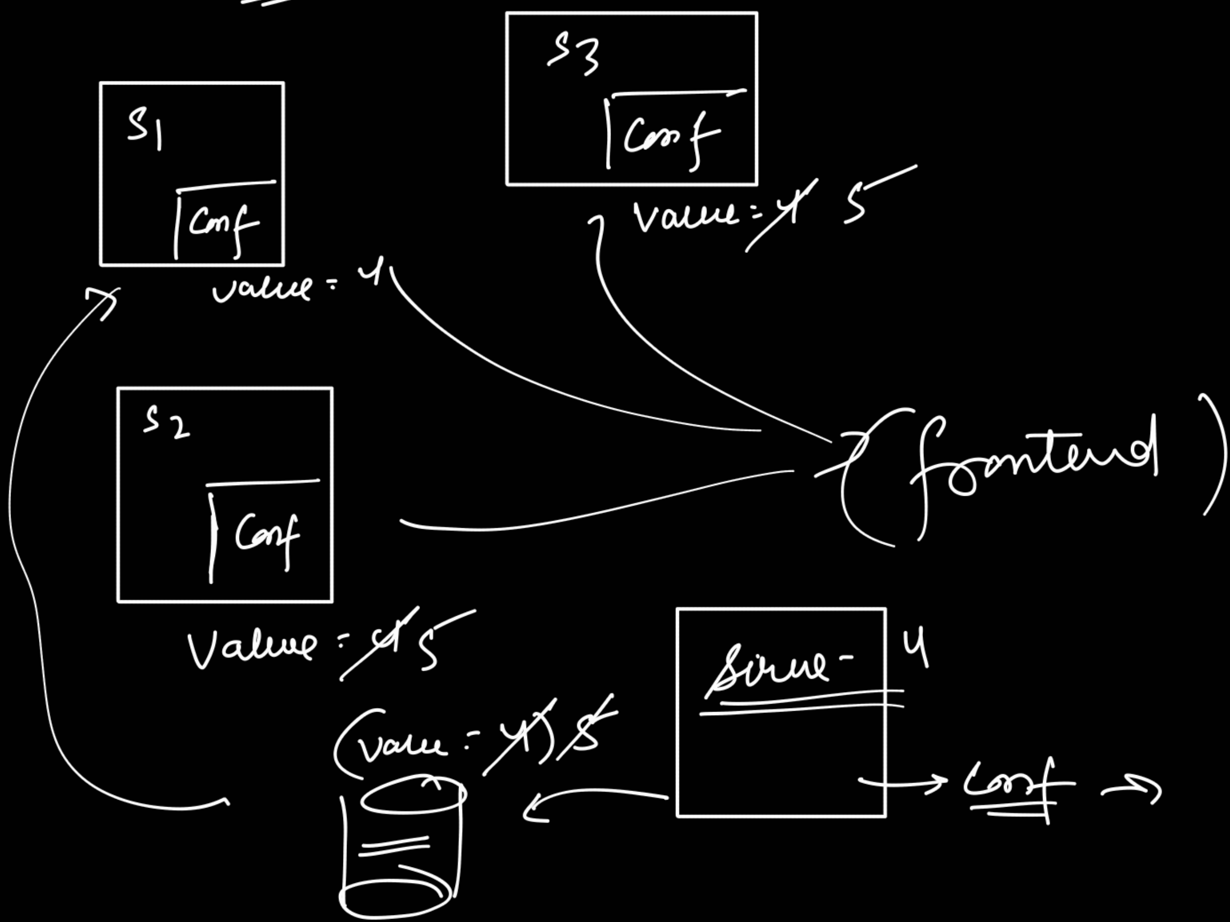
Solution 1) DB



Problem

→ lot of DB calls for the same data.

Solution -2



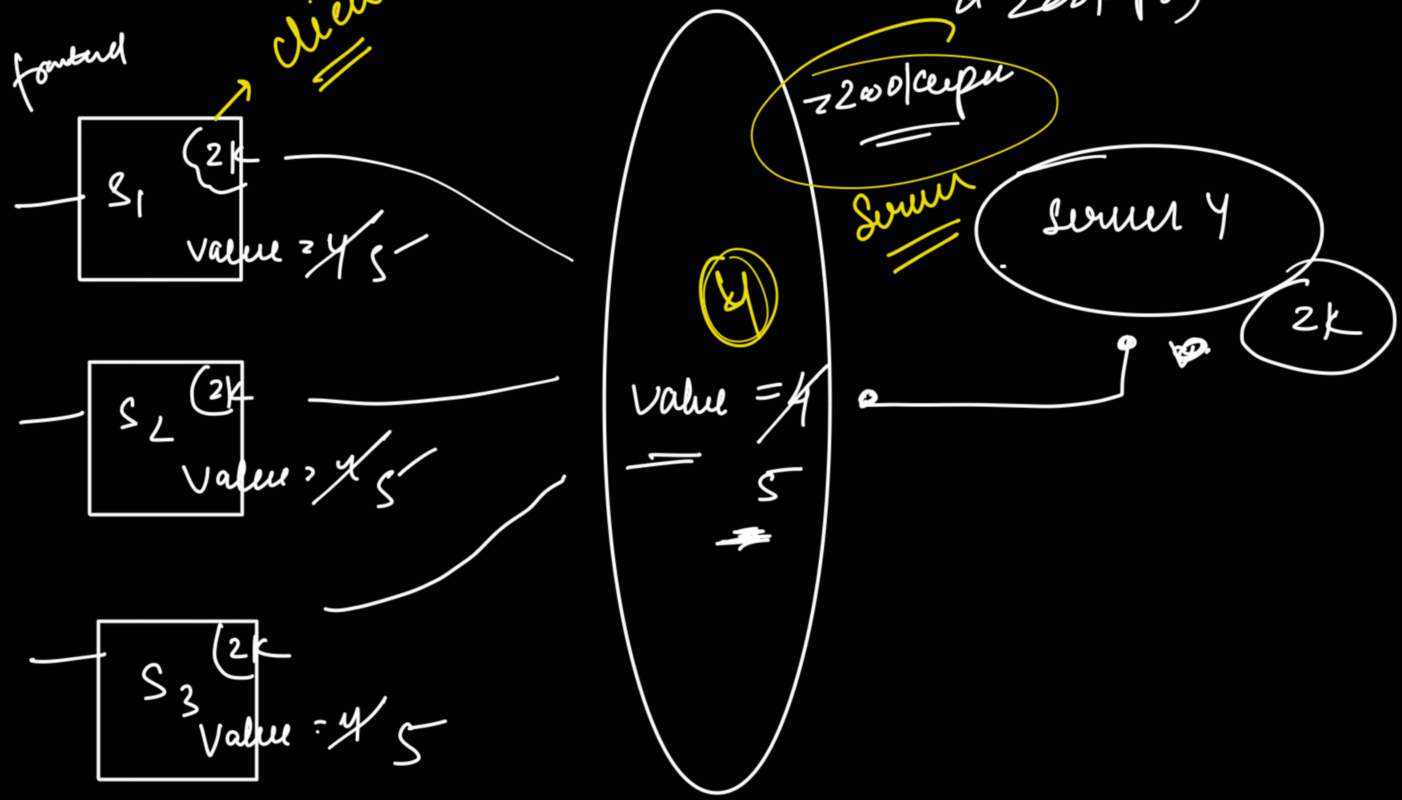
Solution-2 1) DB-trigger

2) webhook

problem :- if one is server al down.

zookeeper :- properties :-

1) FIFO ~~(x)~~
2) storing of data (x)
3) tcp
4) $\boxed{\text{watcher}}$ (callback any time there is a change in a zookeeper)



video-2

Single node zookeeper setup.

→ Download it from website.

→ Study commands

→ (dataDir, ClientPort)

→ Configurations (Database, ...

→ Commands used in video

→ ./bin/zkServer.sh start config/zoo.cfg
→ ./bin/zkServer.sh stop
→ ./bin/zkServer.sh start-foreground
→ ./bin/zkServer.sh status

} Server

→ ./bin/zkCli.sh -server localhost:2181

↳ When client is running

} Client

- → use ls /
- create node/
- → create node/child

Try yourself

---

video 3 Multi Server setup of Zookeeper

→ minimum node 3

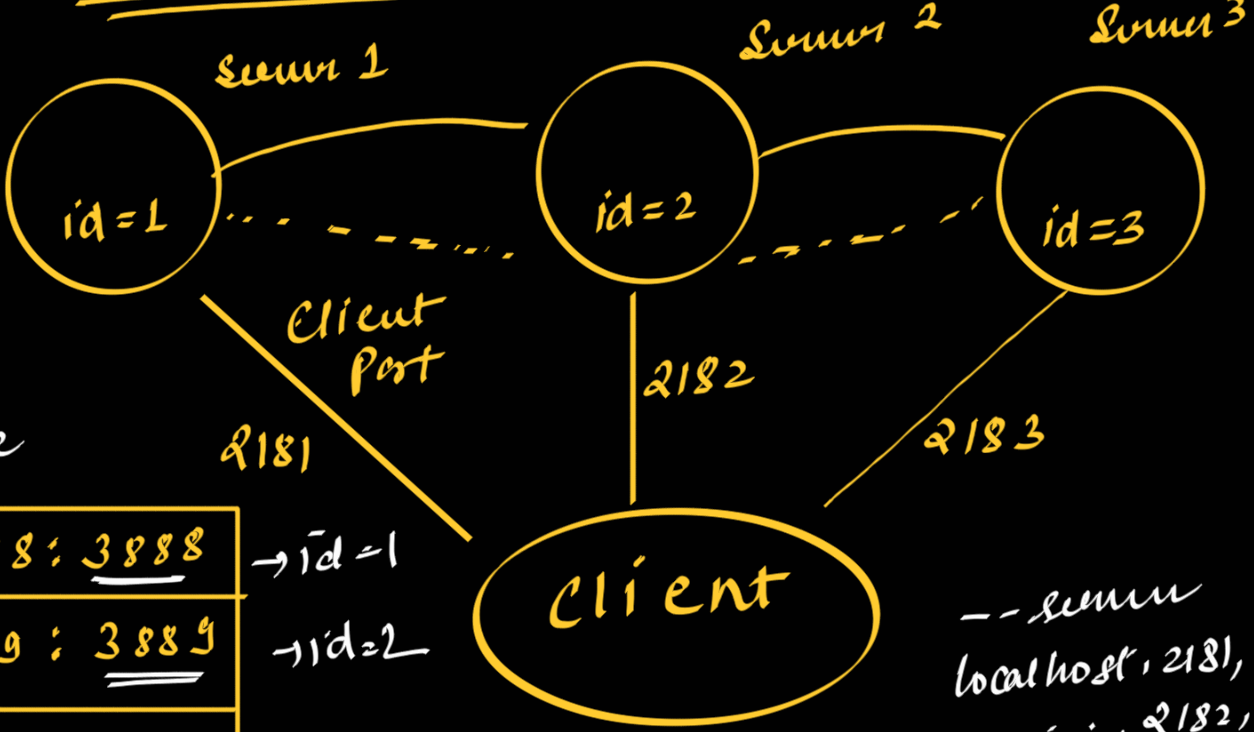
→ recommend odd number of server.

→ quorum   loon ← (1 → leader, 3 → follow)

→ leader-follower node   3 – follow

{leader failed, a new election will happen}

multi Node Setup



table

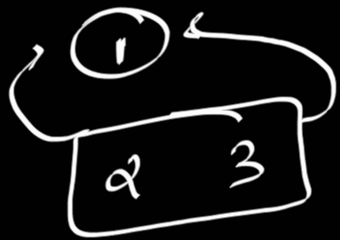
2888 : 3888	→ id=1
2889 : 3889	→ id=2
2890 : 3890	→ id=3

--server
localhost:2181,
..........:2182,
..........:2183.

→ ticktime → 4000 (multiplier)

→ inittime → 1 →

→ synctime → 2 →



time = 1×4000 = 4000 ms
2×4000 = 8000ms

leader-election :-

synctime

winner

→ 1 → election

→ 2 → election } → 1

→ 3 → election } → 2 ×

→ 4 → election } → 4

→ $\boxed{1, 2, 4}$ → (1) → win

(Sync time) → 2 sec

1 → 1 winner

1, 2 → 2 winner

1, 2, 3 → 2 winn

leader

2 — follower 1

2 leader

1 3 follr

2 2 2

2 2 2

[1, 3] → zknode

2

9:10 m 9:10 am

1

→ 9:10:12 sec

9:10:14 sec

9:10:16 sec

3

9:11:13

9:11:15 (zknode)

9:11:12

$\boxed{9:11:17: 0001}$

9:11:17

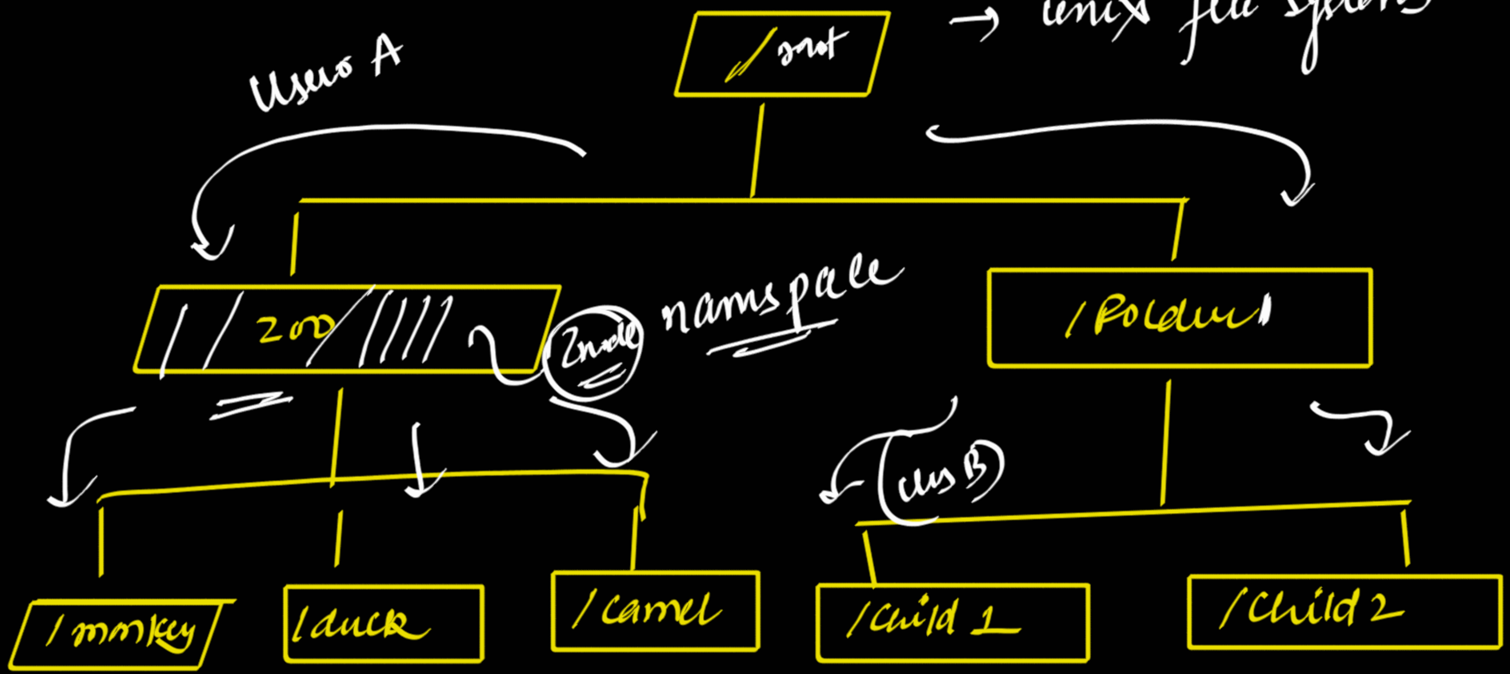
seen 1

② - goes down

times are different

9:10:17 → 9:10:17

Zookeeper Data Model - video-4



Properties

1) namespace unix system
2) each node similar to unix system

[Znode]
→ internal

- Data : string, max 1 mb
- path : /ab123 (·)
- Access control list :
- Stats information : versions

Types of Znode

1) Persistent
2) Ephemeral

{Sequential)

So total types → Persistent non-sequential
Persistent sequential
Ephemeral non-sequential
Ephemeral sequential

Persistent → 1) have life time unless it deleted
2) can have children znode

Ephemeral → 1) session node
2) no children znode

Persistent ⇒ lifetime Children

→ [/A] → Zookeeper serve

→ /A/child1 , /A/child2

Ephemeral

→ session

→ /e → deleted session X

PS — (/A/seq00001 , /A/seq00002) lifetime sequence

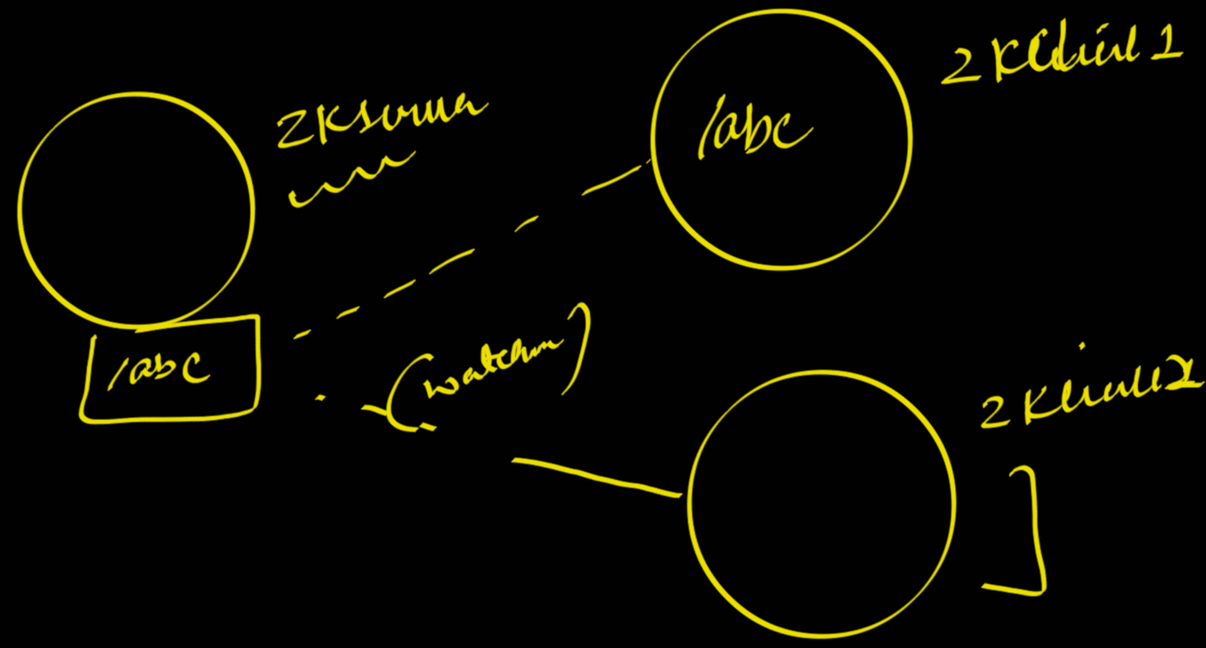
PNS — /A ( ) lifetime

ES — (seqe00009, seqe000010) session sequence

ENS — /E ( ) session

Video 5

Designing ZooKeeper



features:

1) Znode < Persistent / ephemeral
2) Get Znode
3) Delete Znode
4) All Znode

7) Register Znode with client
8) Unregister
9) Context switching

5) Close Client

10) Close all the conn

6) Restart client

Video-6

System Design Problem

Solved with Zookeeper

1) namespacing

/root

/1 /2 /3

/ (videocal)

/1 /2 /3 /4

2) Divide range of servers

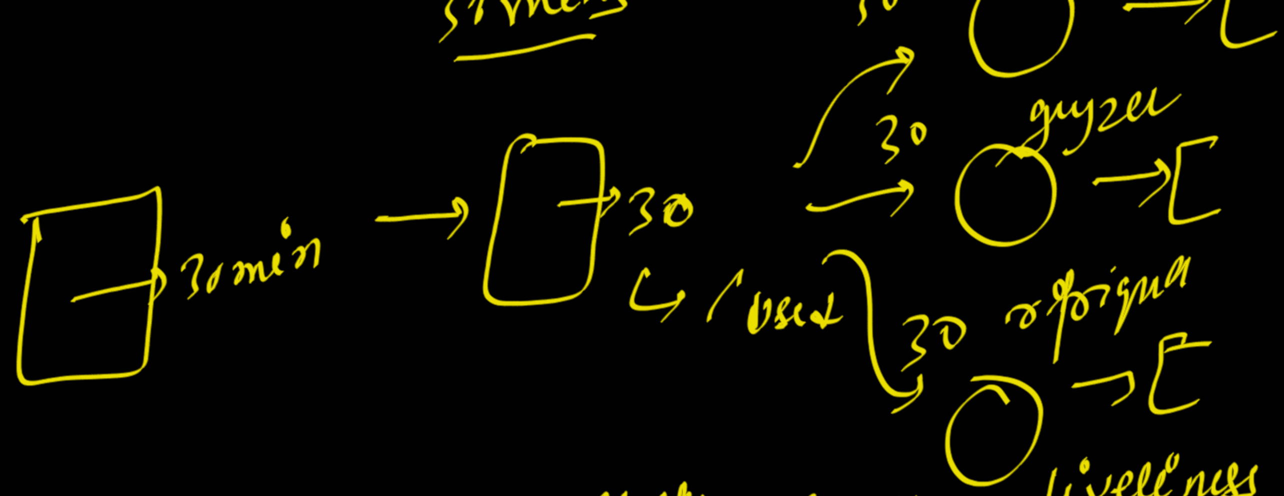
S_1 → [1 - 5000] ⟷ /Partitions/1

S_2 → [5001 - 10000] /Partitions/2

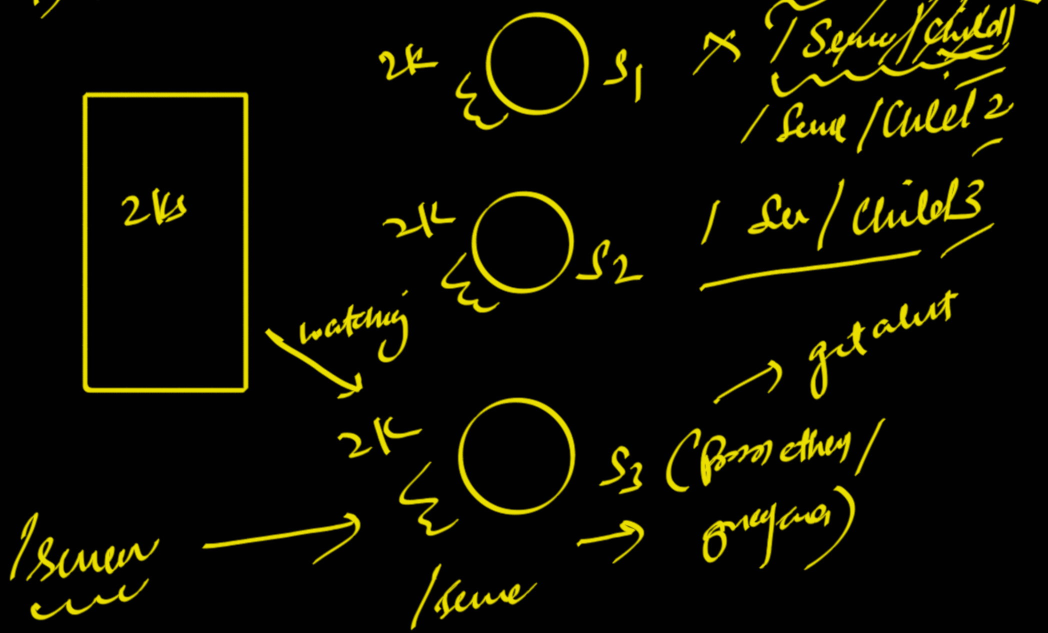
S_3 → [10001 - 15000] /Partitions/3

3) if user is entering the house after

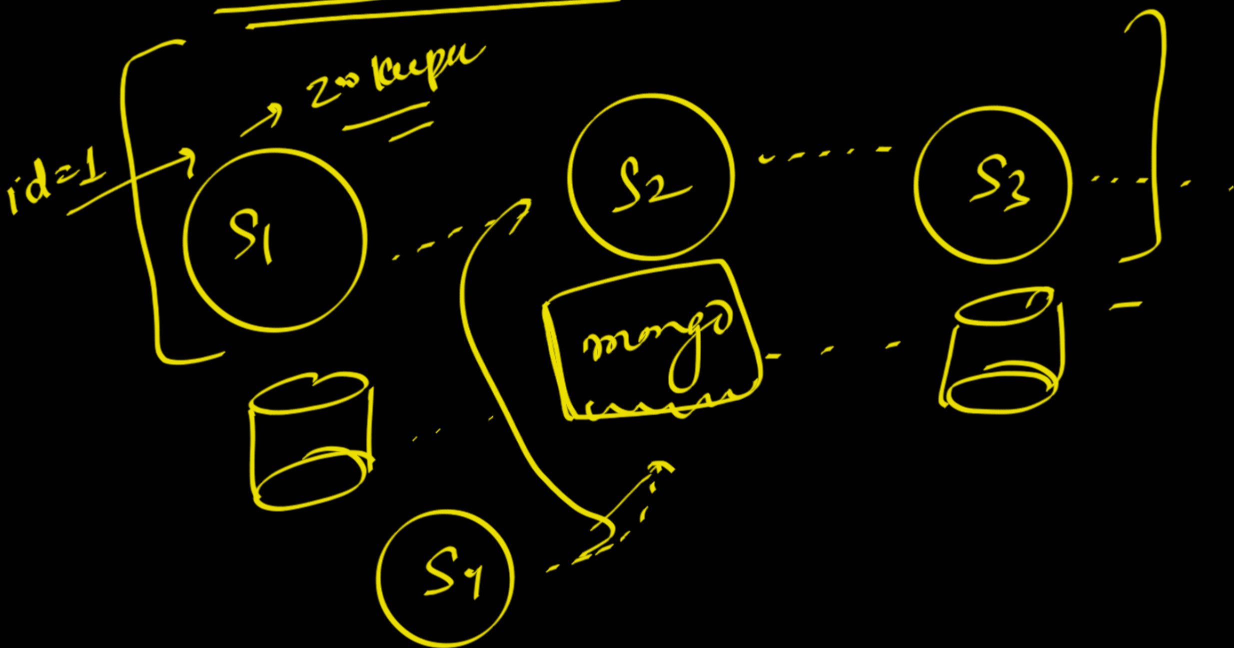
AC



4) zookeeper checks server liveliness



5) Distributed locking



6) Queue

(ZK)

→ /ser/seq 0000 0

→ Sn /seq 0000 /   ↳ /Server/

→ Sn /seq 0002

6) State manager

Ks

/user/seq001/ → 1 instance

→ seq002 → /...

Os ↳ seq003 ... /...

orderid

xyz pqr / Created / Dispatch] ✓

/ Process] ✓